

Problem 2: Hand-on Experience with a Distributed Computation.....	3
Algorithm.....	3
Design.....	5
Data.....	6
5 nodes.....	6
2 nodes.....	6
1 node.....	6
Data Analysis.....	6
Reflection.....	6

Problem 2: Hand-on Experience with a Distributed Computation

Algorithm

For this problem I decided to use apache spark and the page rank distributed algorithm. For the dataset I needed a directed graph so I chose the Live Journal directed web graph from SNAP.

Links are provided below.

<https://spark.apache.org/downloads.html>

I created a python program that took the txt file and translated it into a form that could be processed by spark. Then the program printed out statistics in the terminal. The full web graph consisted of (<https://snap.stanford.edu/data/soc-LiveJournal1.html>) :

LiveJournal is a free on-line community with almost 10 million members; a significant fraction of these members are highly active. (For example, roughly 300,000 update their content in any given 24-hour period.) LiveJournal allows members to maintain journals, individual and group blogs, and it allows people to declare which other members are their friends they belong.

Dataset statistics	
Nodes	4847571
Edges	68993773
Nodes in largest WCC	4843953 (0.999)
Edges in largest WCC	68983820 (1.000)
Nodes in largest SCC	3828682 (0.790)
Edges in largest SCC	65825429 (0.954)
Average clustering coefficient	0.2742
Number of triangles	285730264
Fraction of closed triangles	0.04266
Diameter (longest shortest path)	16
90-percentile effective diameter	6.5

 soc-LiveJournal1.txt	1.08 GB	Plain Text	Today at 6:21 PM
 soc-LiveJournal1.txt.gz	259.6 MB	gzip co...archive	Today at 6:20 PM

Spark executors reserve 300MB for the JVM so to calculate the size we will use we can subtract that. We just need the size - 300MB < file size. We can set the size to 512MB-300MB = 212 MB.

So i took the first 1800000 lines which made the new file size:

```
soc-LiveJournal1-250mb.txt 263.5 MB Plain Text Today at 7:20 PM

Number of edges (unique): 17999996
Number of vertices (unique): 2401266
size of data/soc-LiveJournal1-250mb.txt: 263549257 bytes or
251.34 MB
Number of Spark Partitions: 64
```

Since 251MB > 212MB this satisfies the condition that: “The dataset is larger than the allocated memory of a single executor.” More information about this is in the reflection section. In order to run this we have to start a master program which manages the cluster and worker nodes that each run parts of the program in parallel. We can do this via the code below.

```
./sbin/start-worker.sh spark://Isabelles-MacBook-Air.local:7077
--cores 1 --memory 512mb --webui-port 0
```

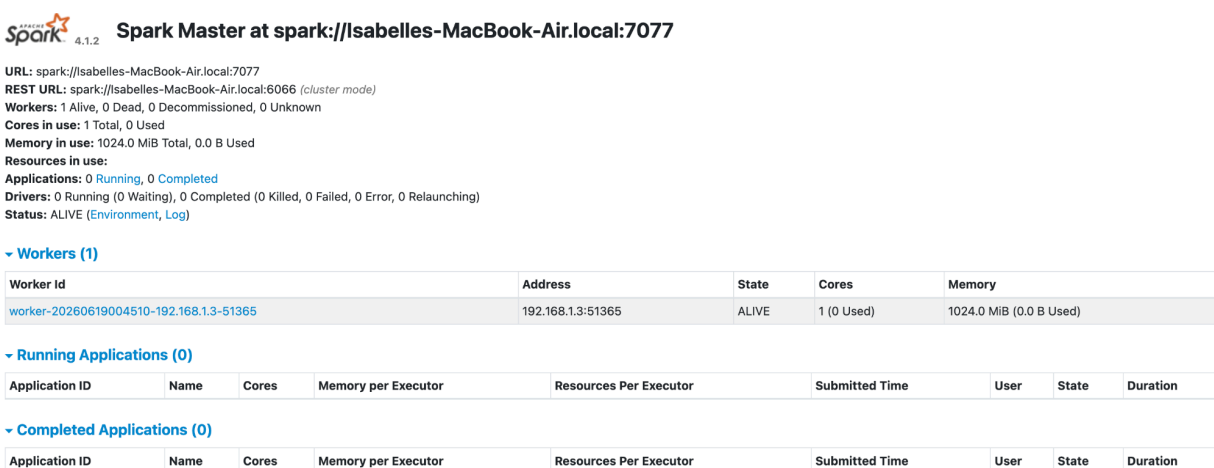
```
found io.graphframes#graphframes-graphx-spark4_2.13;0.11.0 in central
:: resolution report :: resolve 117ms :: artifacts dl 3ms
:: modules in use:
io.graphframes#graphframes-graphx-spark4_2.13;0.11.0 from central in [default]
io.graphframes#graphframes-spark4_2.13;0.11.0 from central in [default]
-----
| conf | modules | artifacts | | | | |
|---|---|---|---|---|---|---|
|      | number | search | dl | evicted | number | dl |
|-----|-----|-----|
| default | 2 | 0 | 0 | 0 | 2 | 0 |
-----
:: retrieving :: org.apache.spark#spark-submit-parent-1593dba0-5a6c-40a3-8d1a-8fe0aa2505a8
confs: [default]
0 artifacts copied, 2 already retrieved (0kB/4ms)
26/06/19 00:31:25 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
```

```
0 artifacts copied, 2 already retrieved (0kB/4ms)
26/06/19 00:31:25 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
/Library/Frameworks/Python.framework/Versions/3.10/lib/python3.10/site-packages/pyspark/sql/classic/dataframe.py:146: UserWarning: D
is an internal property, and will be removed in future releases. Use DataFrame.sparkSession instead.
warnings.warn(
[Stage 1:=====> (14 + 2) / 16]
Ln 56, Col 31 Spaces: 4 UTF-8 LF ( ) Python Pyt
```

```
isabelledarling@isabelles-MacBook-Air spark-4.1.2-bin-hadoop3 % pkill -f org.apache.spark.deploy.worker.Worker
isabelledarling@isabelles-MacBook-Air spark-4.1.2-bin-hadoop3 % ./sbin/start-worker.sh spark://Isabelles-MacBook-Air.local:7077 --cores 1 --memory 1
G --webui-port 0
starting org.apache.spark.deploy.worker.Worker, logging to /Users/isabelledarling/Downloads/spark-4.1.2-bin-hadoop3/logs/spark-isabelledarling-org.a
pache.spark.deploy.worker.Worker-1-Isabelles-MacBook-Air.local.out
starting org.apache.spark.deploy.worker.Worker, logging to /Users/isabelledarling/Downloads/spark-4.1.2-bin-hadoop3/logs/spark-isabelledarling-org.a
pache.spark.deploy.worker.Worker-2-Isabelles-MacBook-Air.local.out
starting org.apache.spark.deploy.worker.Worker, logging to /Users/isabelledarling/Downloads/spark-4.1.2-bin-hadoop3/logs/spark-isabelledarling-org.a
pache.spark.deploy.worker.Worker-3-Isabelles-MacBook-Air.local.out
starting org.apache.spark.deploy.worker.Worker, logging to /Users/isabelledarling/Downloads/spark-4.1.2-bin-hadoop3/logs/spark-isabelledarling-org.a
pache.spark.deploy.worker.Worker-4-Isabelles-MacBook-Air.local.out
starting org.apache.spark.deploy.worker.Worker, logging to /Users/isabelledarling/Downloads/spark-4.1.2-bin-hadoop3/logs/spark-isabelledarling-org.a
pache.spark.deploy.worker.Worker-5-Isabelles-MacBook-Air.local.out
isabelledarling@isabelles-MacBook-Air spark-4.1.2-bin-hadoop3 %
```

The webui port 0 is so that each worker gets a different port. We can also see the allocated memory. To change the number of workers we set a variable in the config file. For the page rank algorithm I set the randomization factor to 0.15 (default) and max iterations to just 2. This is because the default would take a very long time to run on my local computer. We should still be able to see a speedup and see significant stats to analyze regardless.

Below is the spark web ui where we can see all of our workers and current jobs on local host 8080.



Spark Master at spark://Isabelles-MacBook-Air.local:7077

URL: spark://Isabelles-MacBook-Air.local:7077
 REST URL: spark://Isabelles-MacBook-Air.local:6066 (cluster mode)
 Workers: 1 Alive, 0 Dead, 0 Decommissioned, 0 Unknown
 Cores in use: 1 Total, 0 Used
 Memory in use: 1024.0 MiB Total, 0.0 B Used
 Resources in use:
 Applications: 0 Running, 0 Completed
 Drivers: 0 Running (0 Waiting), 0 Completed (0 Killed, 0 Failed, 0 Error, 0 Relaunching)
 Status: ALIVE ([Environment](#), [Log](#))

▼ Workers (1)

Worker Id	Address	State	Cores	Memory
worker-20260619004510-192.168.1.3-51365	192.168.1.3:51365	ALIVE	1 (0 Used)	1024.0 MiB (0.0 B Used)

▼ Running Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

▼ Completed Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

Design

The design of this algorithm is a distributed page rank algorithm via apache spark using graph frames (and graph x in java). We will briefly explain page rank.

Page rank is an algorithm to find the relative importance of websites. This is done though simulating a random web surfer where each link has some probability of being clicked on or a random page being chosen. This is a recursive algorithm and each page rank is determined by the page rank of the websites connected to it. More information can be found online as this is not the core subject of this report.

$$PR(u) = \frac{\lambda}{N} + (1 - \lambda) \cdot \sum_{v \in B_u} \frac{PR(v)}{L_v}$$

In spark we can split the work among different work clusters. Each iteration can be mapReduced and shuffled. The rank is calculated first then shuffles those rank contributions among the networks, then they are summed up to get new page ranks.

There are multiple ways to do this, in this case I choose to use python, import pyspark and use the graph frames library from apache spark. I imported functions from pyspark.sql and since pyspark automatically optimizes the cluster scheduling and compression, it did most of the heavy lifting. Another modification is caching the edges and vertices read from the text file which speeds up the algorithm since the program doesn't have to reread information from the disk. Another aspect of this design is the size of each cluster and the number of partitions. For the number of partitions for the 250MB file and 512MB clusters I choose 15 which splits each part into around 50MB which wont overload the heap but also isnt too much so that the program spends too much time switching.

Another way this could be done is the naive way where all of this is done by hand via Spark's RDD and hadoop reduction. Of course this would mean more coding on my side as I would have to manage the switching, scheduling, etc. But this is unoptimized and has to deal with java overhead.

It is obvious that the former option is faster and less memory intensive as the hard programming is done already. More documentation about spark can be found on their website.

Data

5 nodes

```
warnings.warn("DataFrame constructor is internal. Do not directly use it.")
----- DATA -----
Number of edges (unique): 17999996
Number of vertices (unique): 2401266
size of data/soc-LiveJournal1-250mb.txt: 263549257 bytes or
251.34 MB
Number of Spark Partitions: 15
Total time: 178.58 seconds
----- DONE -----
```

2 nodes

```
warnings.warn("DataFrame constructor is internal. Do not directly use it.")
----- DATA -----
Number of edges (unique): 17999996
Number of vertices (unique): 2401266
size of data/soc-LiveJournal1-250mb.txt: 263549257 bytes or
251.34 MB
Number of Spark Partitions: 15
Total time: 287.00 seconds
----- DONE -----
```

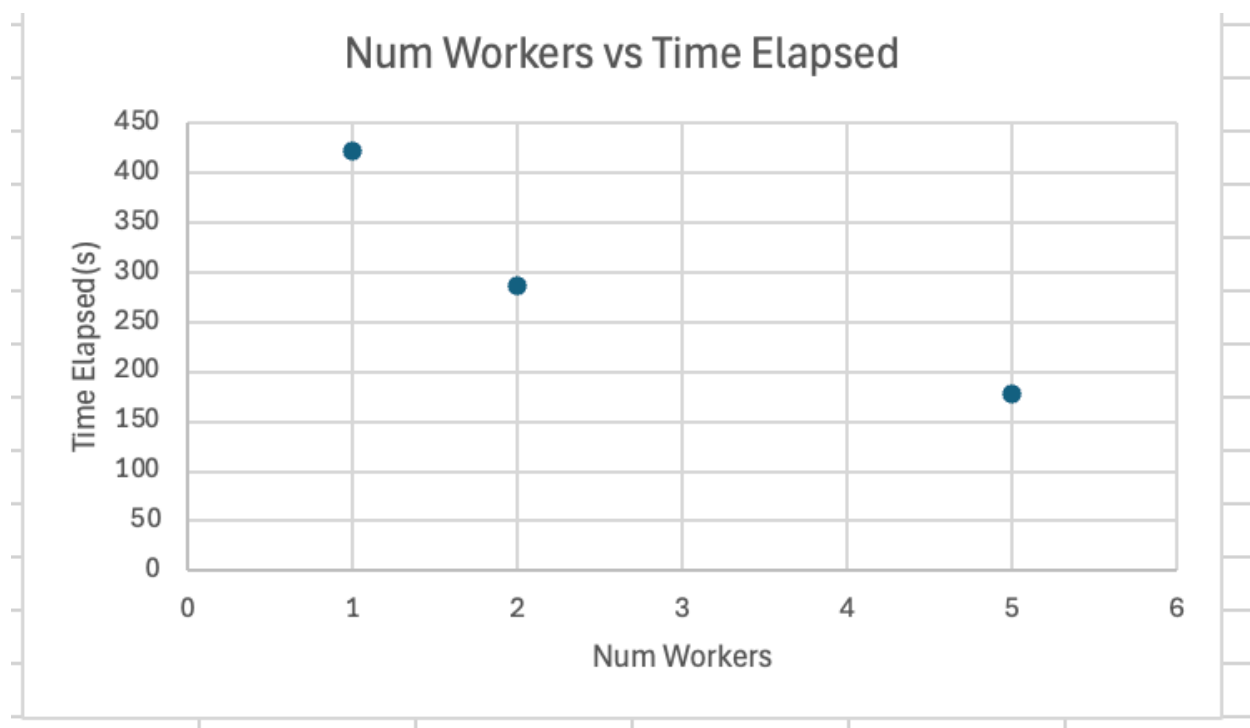
1 node

```
warnings.warn("DataFrame constructor is internal. Do not directly use it.")
----- DATA -----
Number of edges (unique): 17999996
Number of vertices (unique): 2401266
size of data/soc-LiveJournal1-250mb.txt: 263549257 bytes or
251.34 MB
Number of Spark Partitions: 15
Total time: 421.99 seconds
----- DONE -----
```

Completed Applications (7)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260620195249-0006	PageRankWithGraphFrames	1	512.0 MiB		2026/06/20 19:52:49	isabelledarling	FINISHED	7.1 min
app-20260620194723-0005	PageRankWithGraphFrames	2	512.0 MiB		2026/06/20 19:47:23	isabelledarling	FINISHED	4.8 min
app-20260620194340-0004	PageRankWithGraphFrames	5	512.0 MiB		2026/06/20 19:43:40	isabelledarling	FINISHED	3.0 min

Num_workers	Time Elapsed(s)	speedup(from one)	speedup(from previous)	parallel efficiency (E_p)	
1	421.99	0	0	0	
2	287	1.470348432	1.470348432	73.5174216	
5	178.58	2.363030575	1.607122858	47.26061149	



Data Analysis

There is a clear relationship between the number of worker nodes and the time taken to run the program. This is due to the distribution of the work. One node must do everything itself, while two nodes can split the data up. Of course there are diminishing returns as more nodes means more switching overhead which explains the non linear nature of the graph. We also can see the parallel efficiency which is decreased for more nodes which corroborates the diminishing returns.

Reflection

Through this project I learned how algorithms can be distributed and the huge speed benefit in doing this. Using more than one worker and splitting the data and running it in parallel makes the page rank algorithm, and likely most algorithms run significantly faster. I also learned about apache spark, further helping me learn about apache products like solr and nutch and how all of them work together. I also got to work more with hadloop through data outputs and logs. Apache Spark is a distributed processing engine and has built in tools for page rank which is why I choose it for this project. While I didn't work much with page rank, and rather how the algorithm is distributed I did enjoy working with the page rank algorithm. I also liked using pyspark which is apache spark's python library which made it simpler to code.

Through running multiple workers on my local machine, and then running my python algorithm to distribute and control the apache spark output we can see basic statistics about the run.

One problem I ran into was very slow computing times. This is due to the reserved memory in

spark execution nodes and the expansion size of the file once converted into graph frames. The text file more than doubles in size when converted to a text frame and the execution nodes reserve 300MB of memory for the JVM so at first when picking a dataset size I choose a txt file greater than the memory on a single executor but this ran very slow. But once incorporating reserved memory becomes more manageable.

The input for the python file I used and apache spark would be choosing a number of nodes and a directed graph to analyze and the output would be the print out the terminal of the statistics as well as the logs generated by Spark.